

CI / CD

멋쟁이사자처럼 14기 김동민

목차

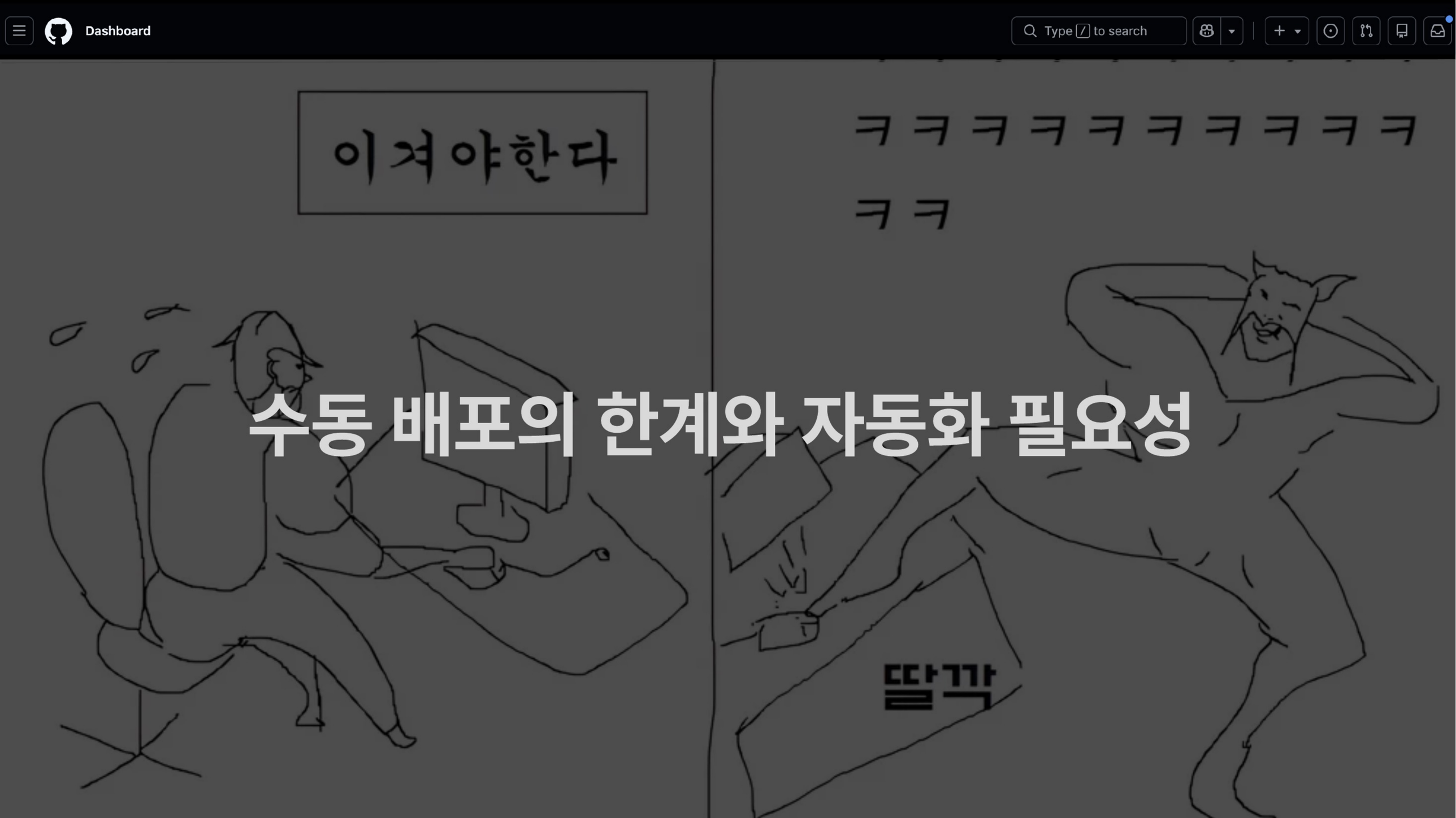
현대 배포의 표준, 도커
도커의 개념
왜 도커일까 ?

실습

수동 배포의 한계와 자동화 필요성
수동배포의 단점
자동배포의 장점

GitHub Action
GitHub Action란 ?
워크플로우의 구성요소
GitHub Secrets

과제


Dashboard
Type / to search
이겨야한다
ㅋㅋㅋㅋㅋㅋㅋㅋㅋㅋ
ㅋㅋ
수동 배포의 한계와 자동화 필요성
딸깍

수동배포

Spring Boot 프로젝트를 서버에 배포하고 코드를 수정한다면



1. GitHub에 push한다.
2. 서버에 SSH로 접속한다.
3. git pull을 한다.
4. 프로젝트를 다시 빌드한다.
5. 기존 서버를 끄고 다시 실행한다.

시간이 많이 들어가고,
실수할 수 있음

수동배포의 단점

반복 작업이 많다

사람이 직접 하기 때문에 실수 가능성이 높다

배포 방식이 사람마다 달라질 수 있다

배포 중 서비스가 중단될 수 있다

⋮

수동 배포는 작은 프로젝트에서는 가능하지만,
서비스가 커질수록 비효율적이고 위험해진다.

자동배포의 장점

자동 배포

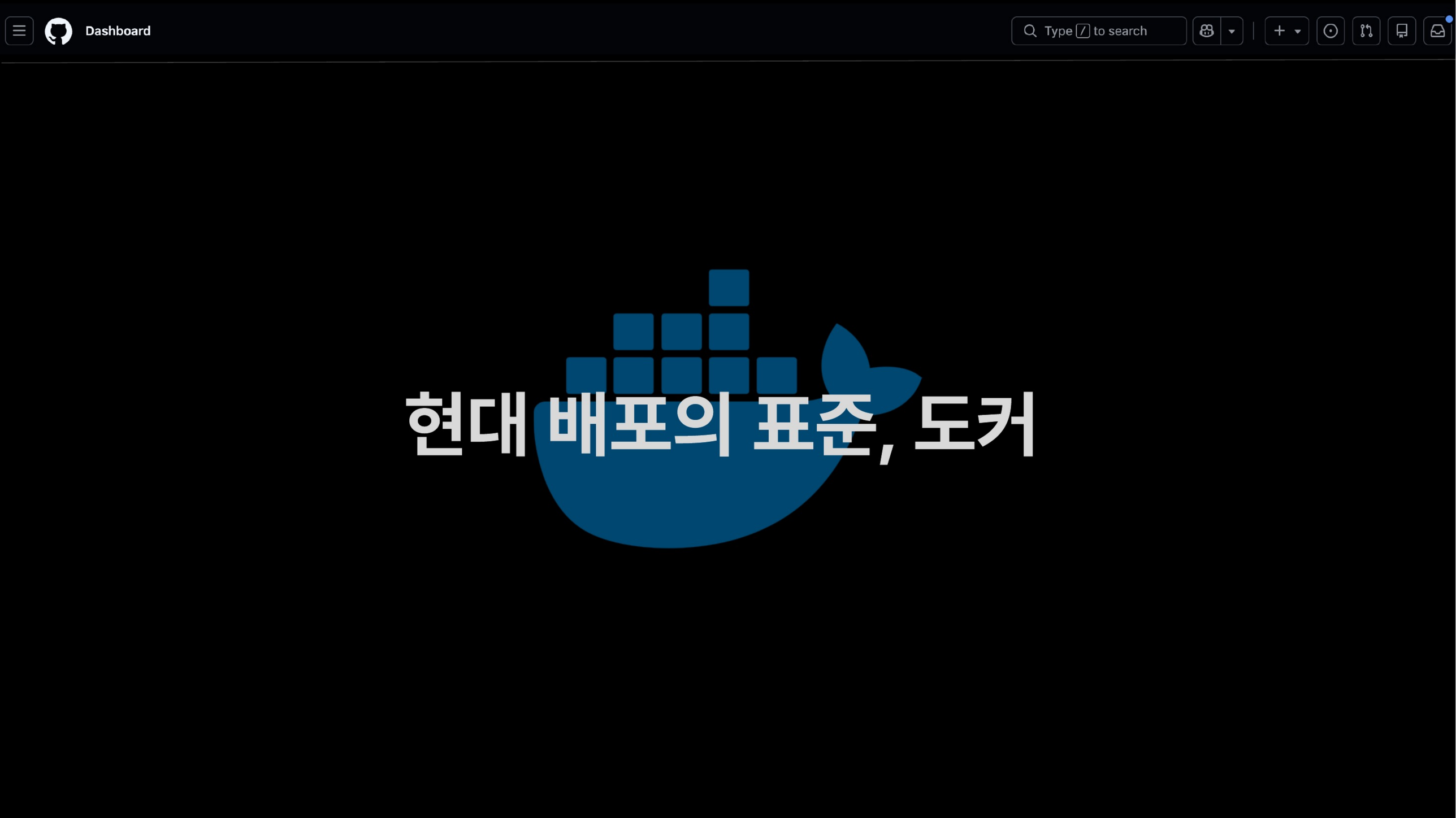
↓

GitHub에 코드를 push하면 정해진 배포 과정이 자동으로 실행되는 방식

- 반복 작업을 줄일 수 있다
- 실수를 줄일 수 있다
- 배포 과정이 표준화된다

Dashboard

Type / to search



도커란 ?

애플리케이션과 실행 환경을 하나로 묶어서,
어디서든 같은 방식으로 실행할 수 있게 해주는 도구이다.



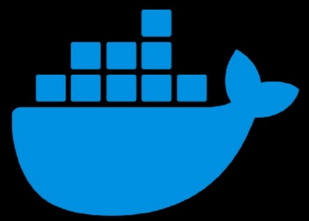
내 컴퓨터

코드



서버

도커란 ?



Dockerfile

Image

Container

Dockerfile ?



도커 이미지를 만들기 위한 설명서

Spring Boot 프로젝트라면 Dockerfile에는

- **Java 17 환경을 사용할 것**
- **빌드된 jar 파일을 이미지 안에 넣을 것**
- **컨테이너가 실행되면 java -jar 명령어를 실행할 것**
- **8080 포트를 사용할 것**

Image ?

애플리케이션 실행에 필요한 것들을 담은 실행 패키지

이미지는 컨테이너를 만들기 위한 설계도 또는 템플릿

Container ?

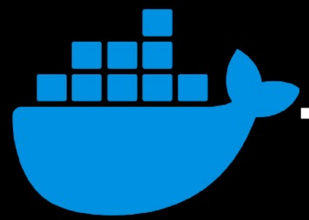
이미지를 실제로 실행한 상태

이미지를 실행하면 컨테이너가 만들어지고, 그 컨테이너 안에서 애플리케이션이 실행된다.

컨테이너 → 독립된 실행 공간

서버 하나에 여러 컨테이너를 띄울 수 있다.

왜 도커일까 ?



환경 차이 문제

Java 버전
빌드된 jar 파일
환경 변수
DB 연결 정보
실행 포트
운영체제 환경
실행 명령어

서버 설정 단순화

sudo apt update
sudo apt install openjdk-17-jdk
git clone ...
cd project
./gradlew clean build
java -jar build/libs/app.jar

CI/CD 연결

GitHub Actions로 자동 배포를 만들 때,
도커를 쓰면 배포 명령이 단순



GitHub Action

GitHub Action



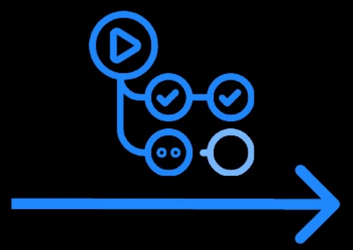
GitHub Actions는 GitHub에서 제공하는 자동화 도구이다.

특정 이벤트가 발생했을 때, 미리 작성해둔 작업을 자동으로 실행해주는 CI/CD 도구

- main 브랜치에 push 되었을 때
- Pull Request가 생성되었을 때
- 특정 시간이 되었을 때
- 수동으로 실행 버튼을 눌렀을 때

GitHub Action

GitHub에 push
↓
SSH 접속
↓
git pull
↓
docker compose down
↓
docker compose up -d --build



GitHub에 push
↓
GitHub Actions 실행
↓
EC2 서버에 SSH 접속
↓
최신 코드 반영
↓
Docker Compose로 재배포

GitHub Action Workflow

GitHub Actions에서 실행되는 자동화 작업 전체

Event - - - - - - - 워크플로우를 실행시키는 조건

↓

Job - - - - - - - 워크플로우 안에서 실행되는 작업 묶음

↓

Runner - - - - - - - job이 실제로 실행되는 컴퓨터 환경

↓

Step - - - - - - - Job 안에서 순서대로 실행되는 각각의 단계

GitHub Secrets

GitHub Actions에서 사용할 민감한 정보를 안전하게 저장하는 공간

비밀번호, SSH 키, 서버 주소

EC2_HOST
EC2_USERNAME
EC2_SSH_KEY
EC2_PORT

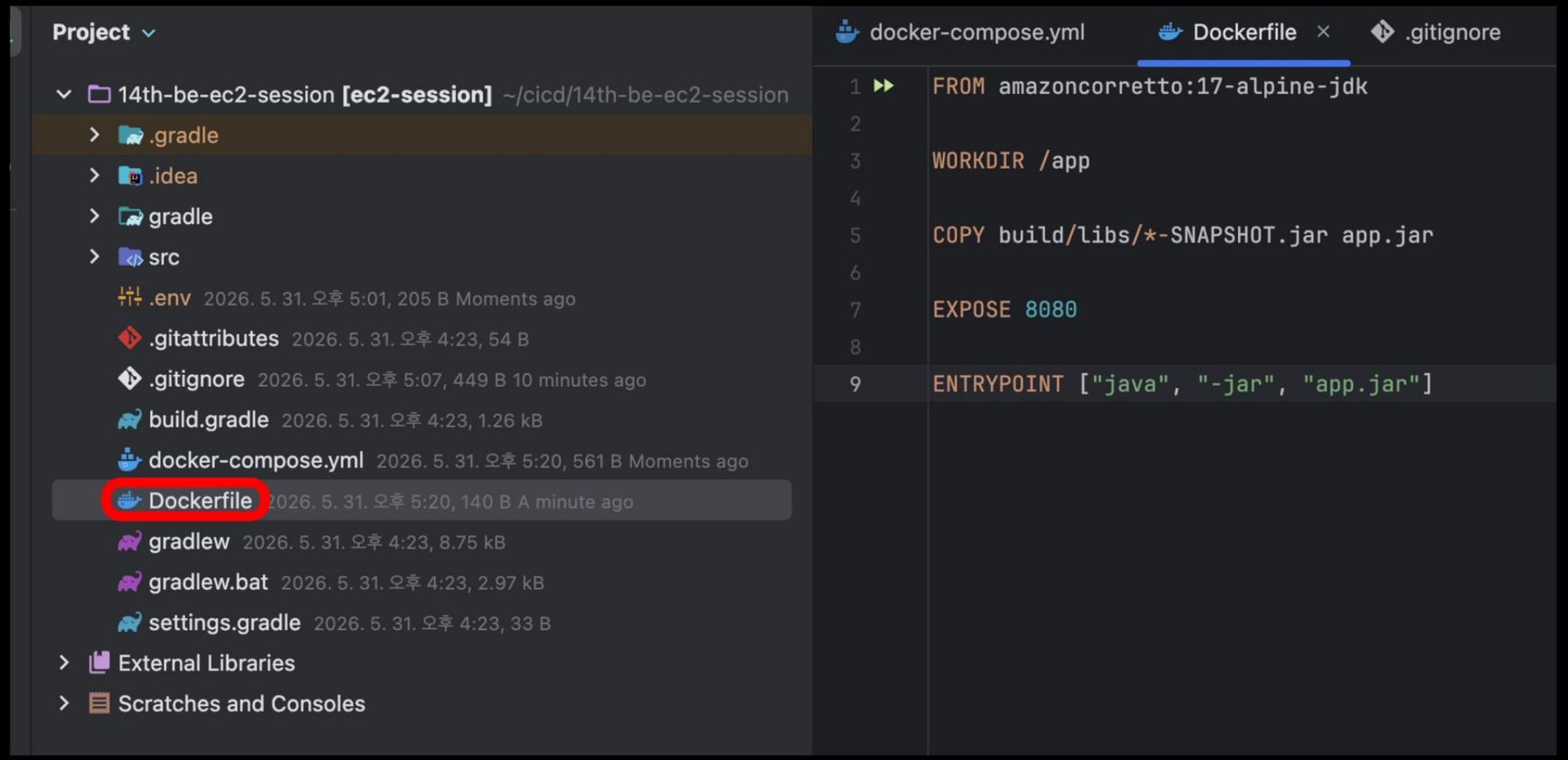


secrets.EC2_HOST
secrets.EC2_USERNAME
secrets.EC2_SSH_KEY
secrets.EC2_PORT



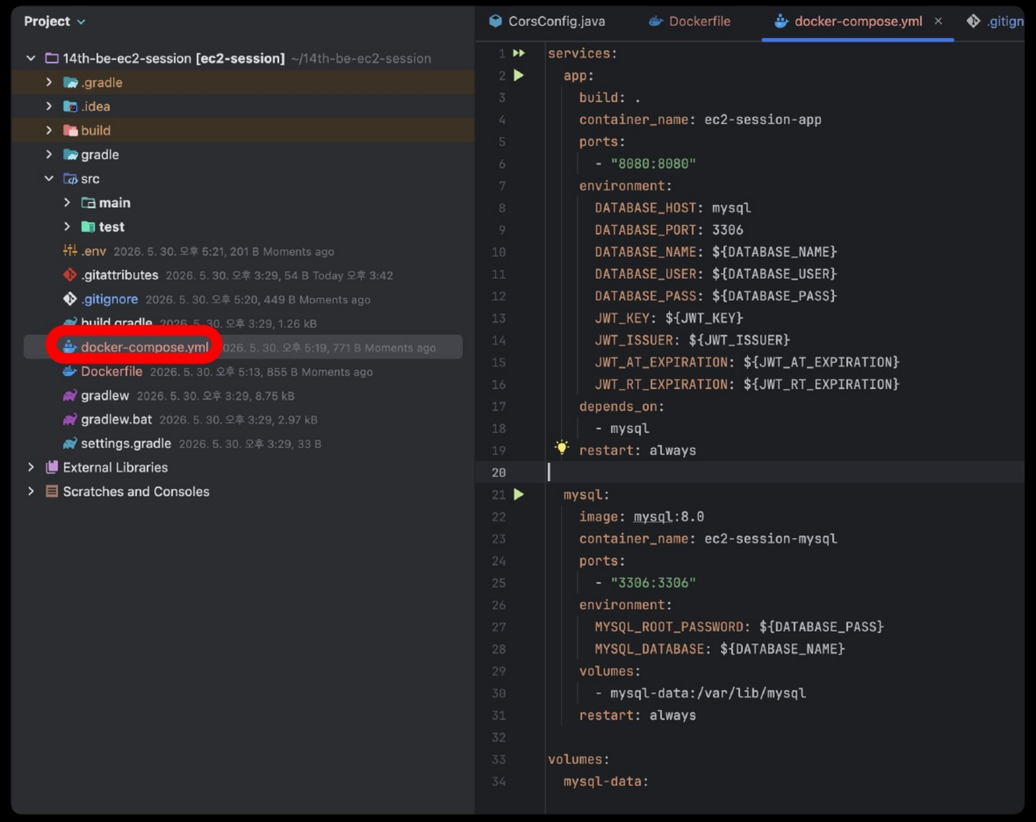
실습

Docker file 작성



docker-compose.yml 작성

컨테이너를 어떻게 실행할지



여러 컨테이너의 실행 설정을 하나의 파일로 관리하는 Docker Compose 설정 파일

포크한 레포지토리에 푸시

```
it@eastminnn 14th-be-ec2-session % git add .
it@eastminnn 14th-be-ec2-session % git commit -m "chore : Docker 배포 설정 추가"
[main d5e901f] chore : Docker 배포 설정 추가
 2 files changed, 64 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 docker-compose.yml
it@eastminnn 14th-be-ec2-session % git push origin main
오브젝트 나열하는 중: 5, 완료.
오브젝트 개수 세는 중: 100% (5/5), 완료.
Delta compression using up to 8 threads
오브젝트 압축하는 중: 100% (4/4), 완료.
오브젝트 쓰는 중: 100% (4/4), 1.16 KiB | 1.16 MiB/s, 완료.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/eastminnn/14th-be-ec2-session.git
   8bb5146..d5e901f  main -> main
```

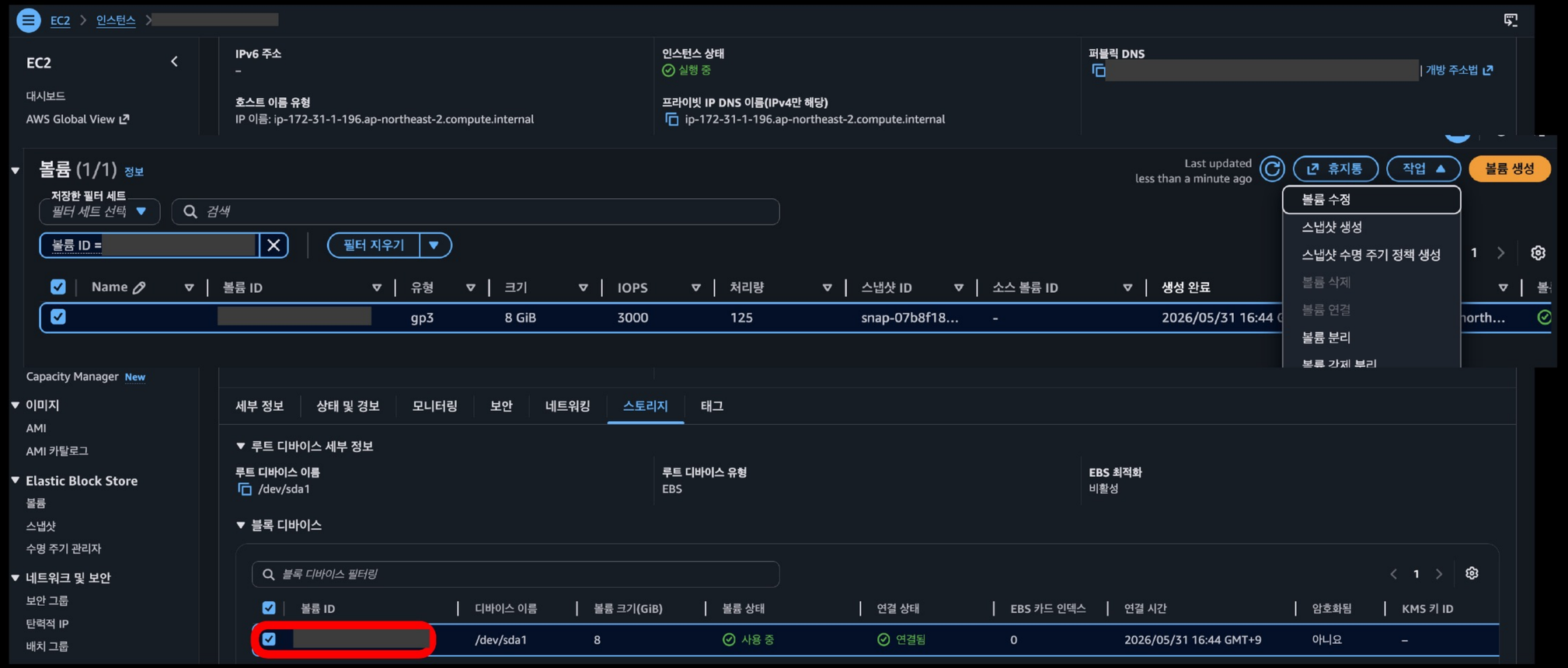
git add .
git commit -m “chore : Docker 배포 설정 추가"
git push origin main

.gitignore 파일에 .env 추가

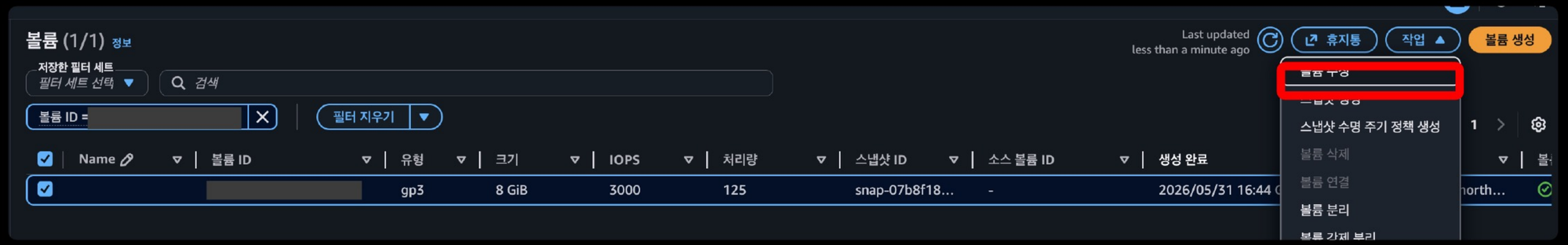
Dashboard

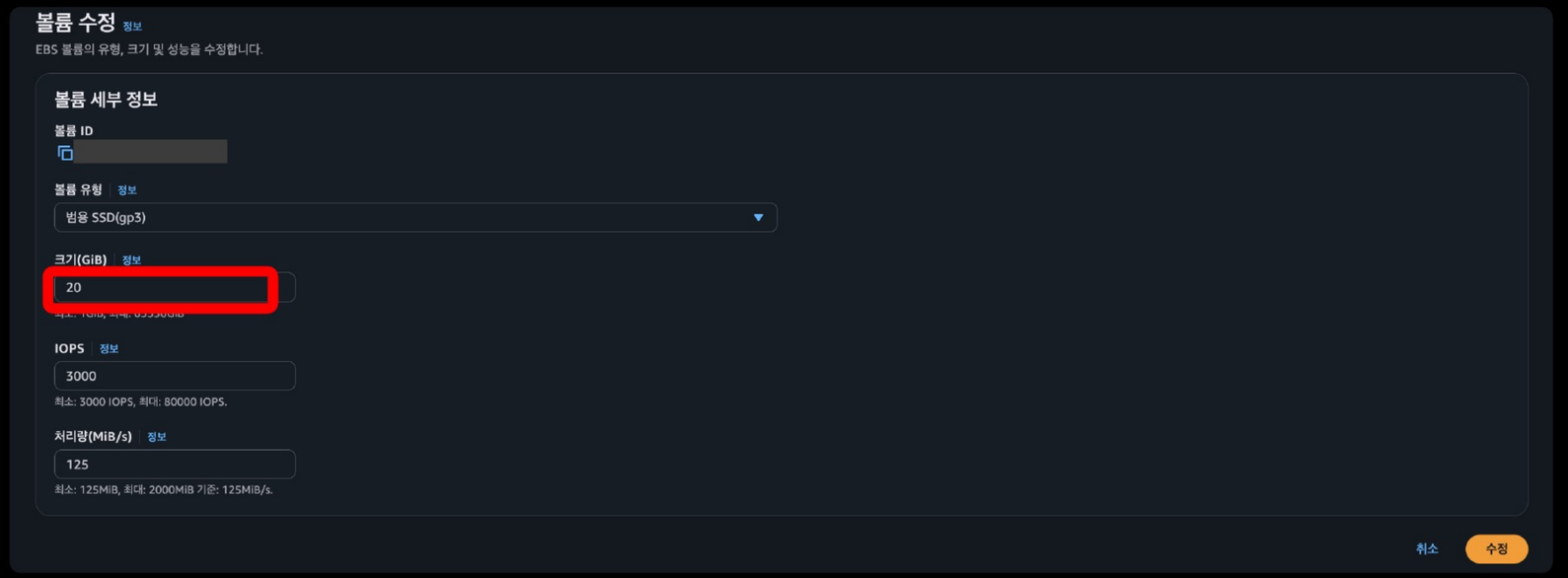
Type / to search

EC2 볼륨 크기 늘리기

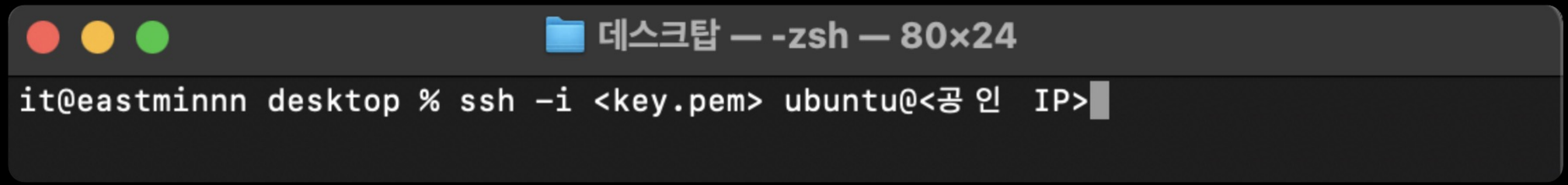


EC2 볼륨 크기 늘리기


볼륨 (1/1) 정보
Last updated
less than a minute ago
휴지통
작업
볼륨 생성
저장한 필터 세트
필터 세트 선택
검색
볼륨 ID =
필터 지우기
스냅샷 수명 주기 정책 생성
볼륨 삭제
볼륨 연결
볼륨 분리
Name
볼륨 ID
유형
크기
IOPS
처리량
스냅샷 ID
소스 볼륨 ID
생성 완료
gp3
8 GiB
3000
125
snap-07b8f18...
-
2026/05/31 16:44


볼륨 수정 정보
EBS 볼륨의 유형, 크기 및 성능을 수정합니다.
볼륨 세부 정보
볼륨 ID
볼륨 유형 정보
범용 SSD(gp3)
크기(GiB) 정보
20
IOPS 정보
3000
최소: 3000 IOPS, 최대: 80000 IOPS.
처리량(MiB/s) 정보
125
최소: 125MiB, 최대: 2000MiB 기준: 125MiB/s.
취소
수정

SSH로 EC2 접속



EC2 볼륨 크기 늘리기

```
ubuntu@ip-172-31-1-196:~/14th-be-ec2-session$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        19G  5.5G   13G  30% /
tmpfs           476M     0  476M   0% /dev/shm
tmpfs           191M  932K  190M   1% /run
tmpfs           476M   32K  476M   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/xvda13     989M   96M  826M  11% /boot
/dev/xvda15     105M  6.3M   99M   7% /boot/efi
none            1.0M     0  1.0M   0% /run/credentials/systemd-networkd.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
none            1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyS0.servic
e
tmpfs            96M  8.0K   96M   1% /run/user/1000
```

lsblk
sudo growpart /dev/xvda 1
sudo resize2fs /dev/xvda1
df -h

도커 설치

```
[ubuntu@ip-172-31-12-210:~$ sudo apt install docker.io -y
Installing:
  docker.io

Installing dependencies:
  bridge-utils  dns-root-data  pigz  ubuntu-fan
  containerd    dnsmasq-base   runc

Suggested packages:
  ifupdown         | cgroup-lite  docker-compose-v2  rootlesskit
  aufs-tools       debootstrap    docker-doc          zfs-fuse
  cgroupfs-mount   docker-buildx  rinse               | zfsutils

Summary:
  Upgrading: 0, Installing: 8, Removing: 0, Not Upgrading: 39
  Download size: 74.0 MB
  Space needed: 281 MB / 4689 MB available
```

sudo apt install docker.io -y

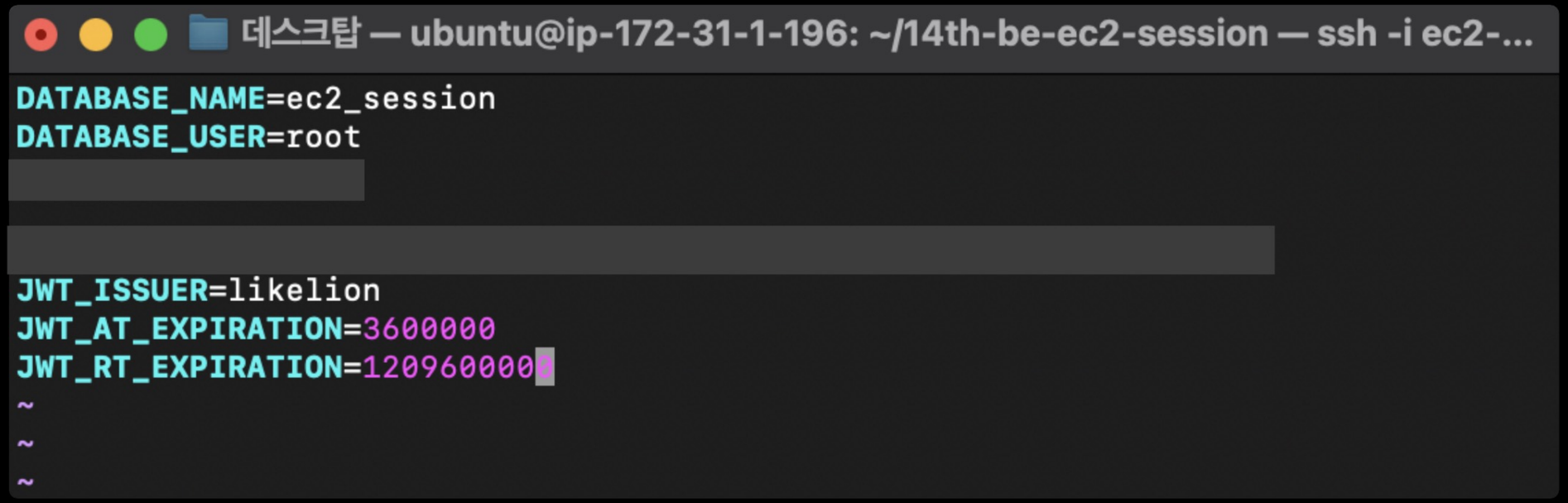
sudo systemctl start docker

sudo systemctl enable docker

sudo apt install docker-compose-plugin -y

안되면 → sudo apt install docker-compose-v2 -y

환경변수 설정



cd ~/14th-be-ec2-session
vim .env

Docker compose 실행 (수동 도커 배포)

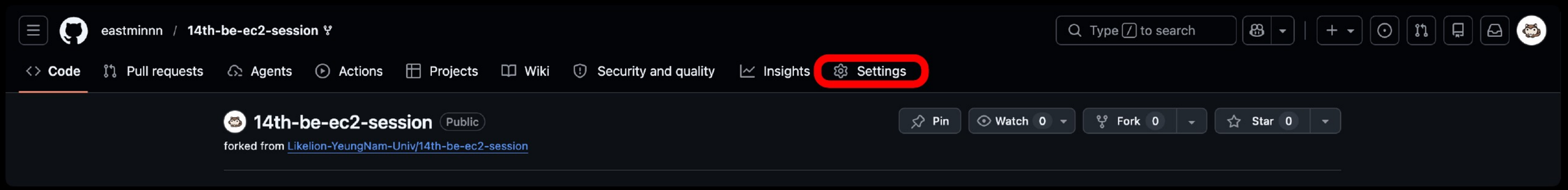
```
[+] Running 5/5
 ✔ app                                        Built                                0.0s
 ✔ Network 14th-be-ec2-session_default        Created                              0.2s
 ✔ Volume 14th-be-ec2-session_mysql-data      Created                              0.0s
 ✔ Container ec2-session-mysql                Started                              0.9s
 ✔ Container ec2-session-app                  S...                                 1.1s
ubuntu@ip-172-31-1-196:~/14th-be-ec2-session$ sudo docker compose logs -f app
```

chmod +x gradlew

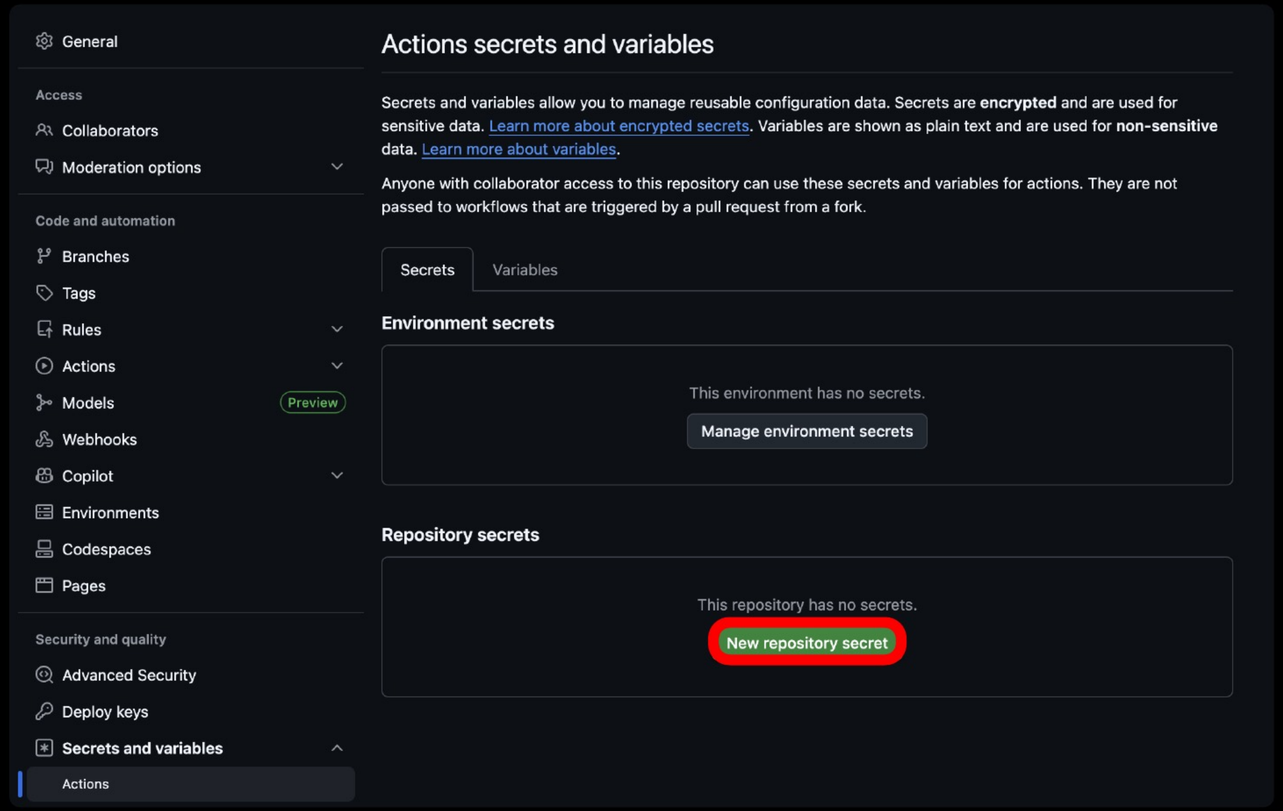
./gradlew clean bootJar -x test

sudo docker compose up -d --build

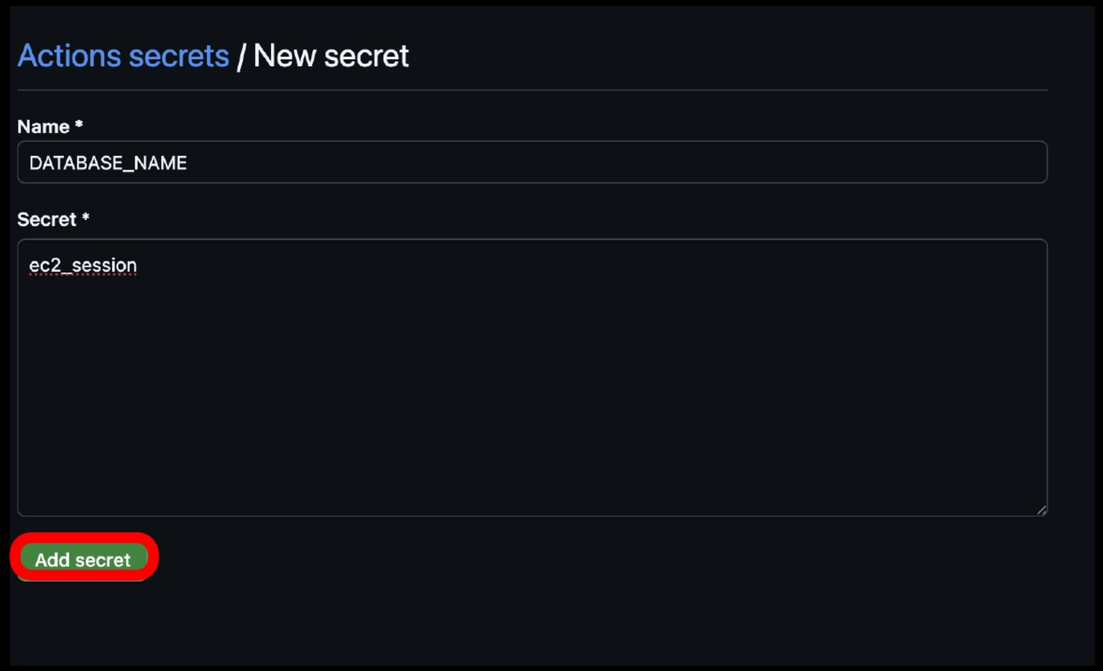
GitHub Secrets 등록



GitHub Secrets 등록



GitHub Secrets 등록



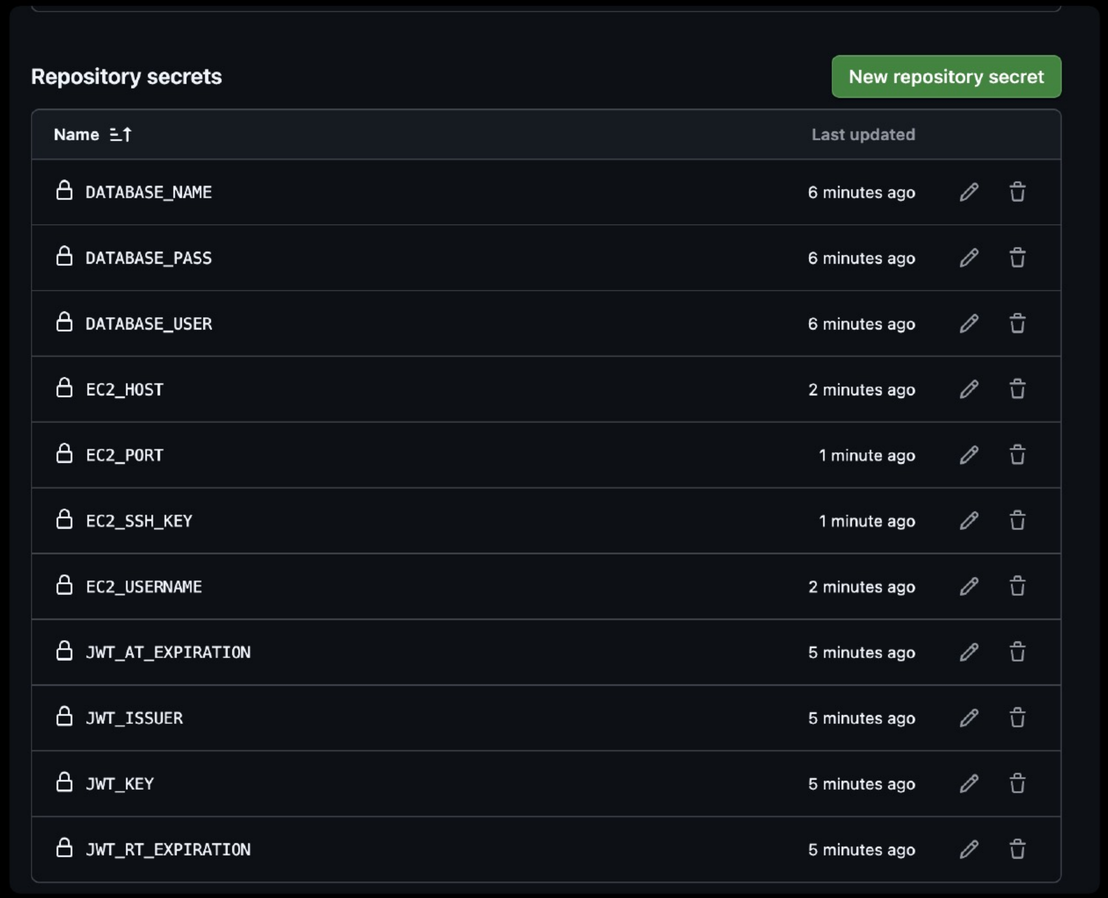
EC2_HOST → 공인 IP

EC2_USERNAME → ubuntu

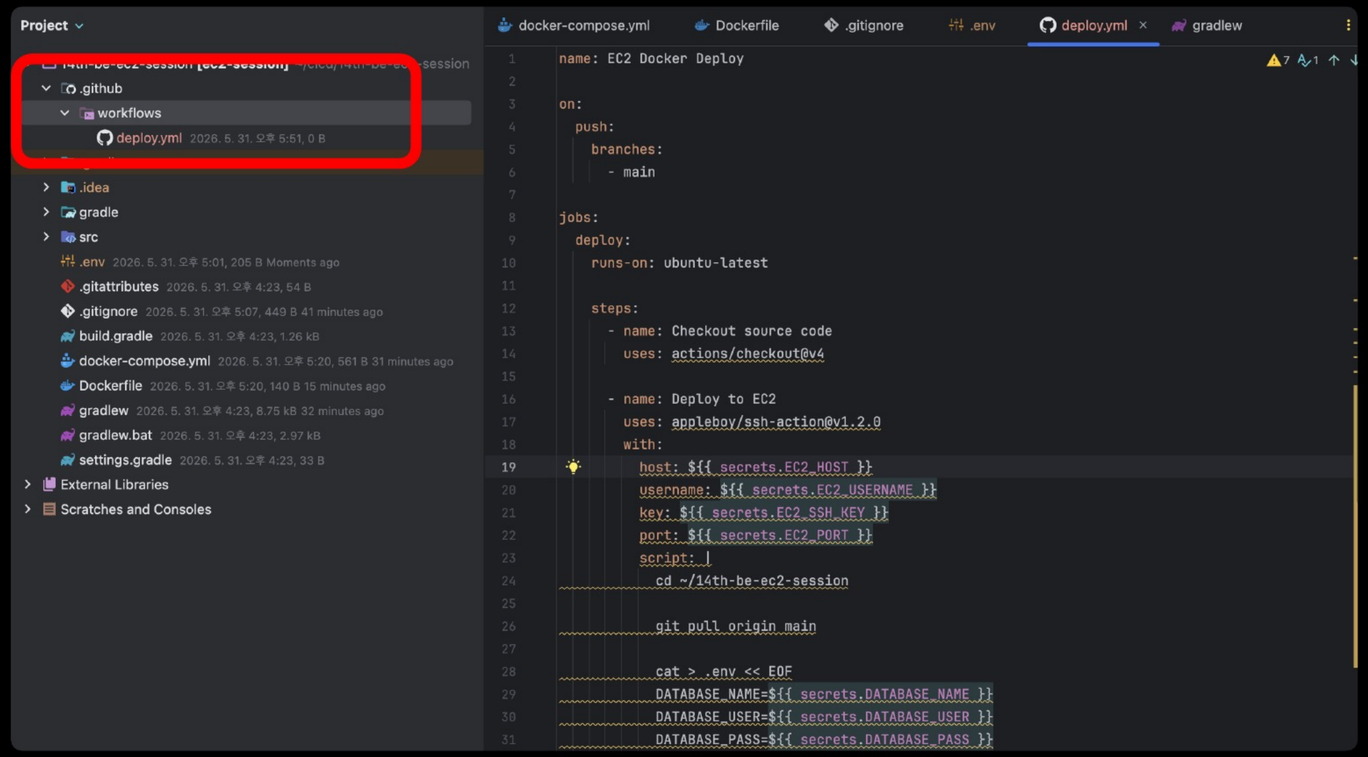
EC2_SSH_KEY →ec2-session-key.pem 전체 내용

EC2_PORT →22

GitHub Secrets 등록

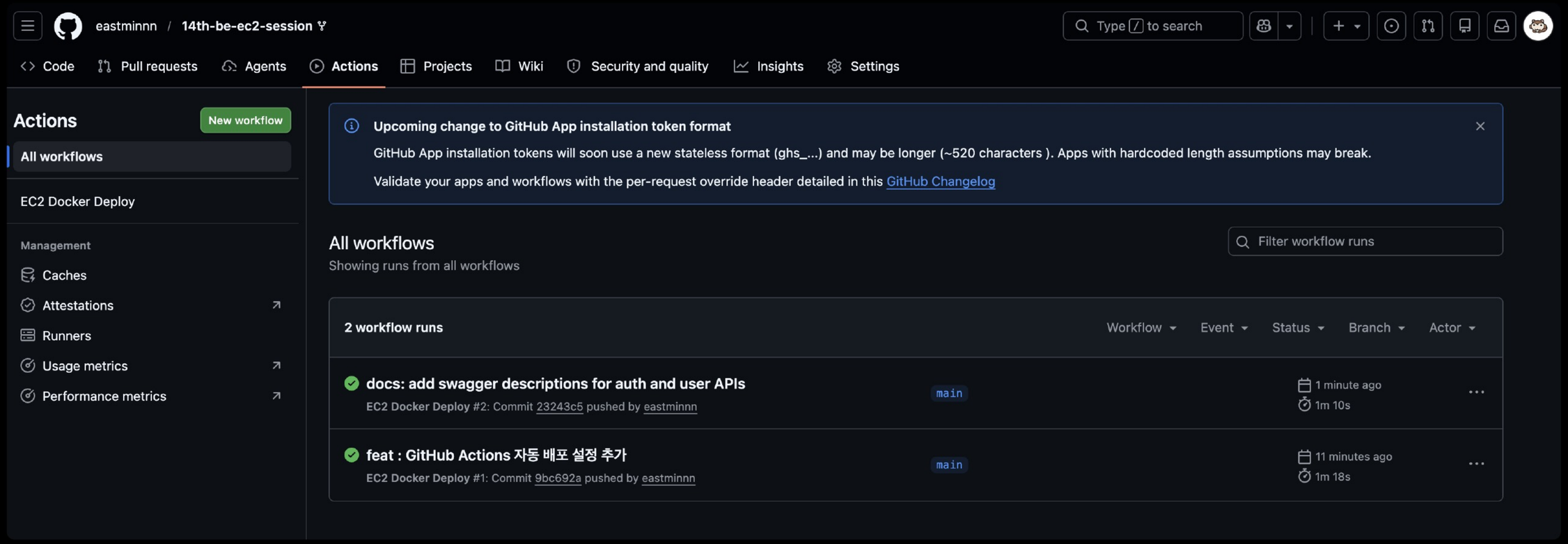


GitHub Actions workflow 파일 작성




```
git add .github/workflows/deploy.yml
git commit -m "GitHub Actions 자동 배포 설정 추가"
git push origin main
```

GitHub Actions workflow





과제

Swagger 설정 (Controller, Request 상세히)

레포지토리 주소, 도메인 주소 카톡으로 보내주세요

과제 기한 : 6월 7일